



STUDENT

0010-RBX

TENTAMEN

250818 DV2606 2110

Salstentamen (DO)

Kurskod	--
Bedömningsform	--
Starttid	18.08.2025 09:00
Sluttid	18.08.2025 14:00
Bedömningsfrist	--
PDF skapad	05.11.2025 17:23
Skapad av	Anders Göransson

1.1 The cpu execution time can be described as $T_{exe} = I * CPI * T_c$. Which of the the following alternatives lower the CPI, thus increasing the performance?

☒ Increase the instruction-level parallelism.



☐ Reduce the cache miss latency.

☐ Divide the instruction execution into more and smaller pipeline stages.

☐ Increase the cache hit ratio.

☐ All alternatives are correct.



Fel. 0 av 1 poäng.

1.2 When we parallelize a sequential program, we find that we only can parallelize 95% of the program, i.e., 5% of the sequential execution time cannot be executed in parallel. What is true about the parallel program? (assuming the same input size, etc.)

☐ None of the alternatives is correct.

☐ The parallel program can only run on a shared-memory multiprocessor.

☒ The parallel program can run at most 20 times faster than the sequential program.



☐ The speedup will be super-linear

☐ The parallel program will be 95% faster than the sequential program.

Rätt. 1 av 1 poäng.

1.3 What is the (asymptotic) parallel time complexity of *matrix-matrix multiplication* using n^2 processors, assuming $n \times n$ matrices?

- ☐ $O(n \log n)$
- ☐ None of the alternatives is correct.
- ☐ $O(\log n)$
- ☐ $O(n)$
- ☒ $O(n^2)$



Fel. 0 av 1 poäng.

1.4 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

- ☐ Increase the number of processors.
- ☐ Increase the clock frequency of the processors.
- ☐ Put more barrier synchronizations in the code.
- ☒ All alternatives decrease the efficiency.
- ☐ Decrease the input data set size.



Rätt. 1 av 1 poäng.

1.5 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *increase* the efficiency?

- ☐ Increase the number of processors.
- ☐ None of the alternatives increases the efficiency.
- ☐ Put more barrier synchronizations in the code.
- ☒ Increase the input data set size.
- ☐ Decrease the input data set size.



Rätt. 1 av 1 poäng.

1.6 Simultaneous multithreading is ...

- ☐ a software approach to parallelize an application
- ☒ a technique to exploit parallelism in the instruction execution pipeline
- ☐ a technique to implement multi-level caches
- ☐ none of the alternatives is correct
- ☐ a technique to improve branch prediction



Rätt. 1 av 1 poäng.

1.7 MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ None of the alternatives is correct.
- ☒ The cache replies with the requested block, and the block is marked as Shared. 
- ☐ The cache replies with the requested block, and the block stays in state Modified.
- ☐ The request is denied since the block is in state Modified.
- ☐ The processor is always stalled until the bus request is handled.

Rätt. 1 av 1 poäng.

1.8 A fundamental difference between a directory-based cache coherence protocol and a snoopy-based cache coherence protocol is ...

- ☐ A directory-based protocol relies on a shared broadcast medium.
- ☐ All of the alternatives are correct.
- ☐ A directory-based protocol can only be implemented using the write-update policy.
- ☒ A directory-based protocol keeps exact information in which caches copies of each memory block reside. 
- ☐ A directory-based protocol can only be implemented using the write-invalidate policy.

Rätt. 1 av 1 poäng.

1.9 Which alternative is correct about memory consistency models?

- ☒ A memory consistency model always relies on specific synchronization operations 
- ☐ A memory consistency model is a specification of how the hardware handles the ordering of read and write operations. 
- ☐ None of the alternatives is correct.
- ☐ A memory consistency model always guarantees a unique ordering of read and write operations.
- ☐ A memory consistency model always guarantees that the last written value to a variable is returned on a read operation.

Fel. 0 av 1 poäng.

1.10 In a MIMD parallel computer, the different processors (or processor nodes) are connected through an *interconnection network*. Which of the following metric is used when evaluating the performance of an interconnection network?

- ☐ Memory capacity
- ☐ None of the mentioned metrics is valid evaluation metrics
- ☒ Diameter 
- ☐ CPU clock frequency
- ☐ Disk read bandwidth

Rätt. 1 av 1 poäng.

1.11 How many neighbors does each internal node have in a 2-dimensional mesh network with $n \times n$ nodes, i.e., in total n^2 nodes?

☐ $n \log n$

☐ n^2

☐ n

☐ $\log n$

☒ 4



Rätt. 1 av 1 poäng.

1.12 A PRAM (Parallel Random Access Machine) can be useful for ...

☐ Very easy implementation of parallel hardware.

☐ Running applications with very large input data sets.

☐ Designing and evaluating parallel algorithms.



☐ Running message-passing applications with GPU support.

☒ All of the alternatives are correct.



Fel. 0 av 1 poäng.

- 2.1** Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

UMA, uniform memory access, meaning each processor has equal uniform bandwidth and latency. In UMA, each processor has access to a shared memory. Preferable for small-scale systems

NUMA, non-uniform memory access, means each processor has its own private memory. Requires message-passing between processors. Has a high scalability, as each processor works independently and can therefore be added to the system with its own memory. Preferable for large-scale systems.

Ord: 68

Besvarad.

- 2.2** Explain why we need to *protect shared data* in shared memory systems. Give an example what could happen if we do not protect shared data. Discuss some common ways to protect shared data in shared memory systems.

Shared data is important to protect due to multiple processors or threads contending for the same shared data, in turn needing protection measurements such as semaphores or mutex locks to ensure the consistency and correct values of the data, as to prevent outdated values or incorrect computations due to not securing the shared data.

Without protective measurements, data can be overwritten or incremented incorrectly, in turn leading to incorrect computations and/or results.

Some ways to prevent this is by using semaphores, barriers or mutex locks to e.g. make sure the shared data is distributed to one thread/processor at a time.

Ord: 100

Besvarad.

- 2.3** *Cache coherence* is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)
Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)
Describe some advantage and disadvantage with each of the approaches. (1p)

Cache coherence is about each processor having an up-to-date values of the shared memory. It is important to ensure since without cache coherence, outdated values could be used instead of the correct values, in turn leading to using outdated or incorrect values instead of the correct ones.

Each processor may have a local copy of a global variable. When one processor updates its copy, all other processors copy of said variable are...

Write-invalidate: ...changed to be invalid. Advantage is that one processor having the correct value instantly, while the drawback is when the other processors need to use that variable, they need to go fetch the new value from the first processor.

Write-update: ...immediately updated to the new value. Advantage is that each processor now has the new value of the variable accessible at one, but the drawback is that it can become a serialized bottleneck (due to high memory traffic) when each processor is getting its variable value updated.

Ord: 160

Besvarad.

- 2.4** When we parallelize a sequential application, e.g., by using threads or message-passing, so it can execute in parallel on N processors, we often find that the parallel program is not N times faster. Describe two important reasons for that.

One reason can be explained by Almdahls law, which states that some parts of the program cant be parallelized, meaning these sequential parts essentially become a sequential bottleneck for the program.

Another reason can be explained by the drawbacks of parallelization, as when parallelizing a program, to ensure the same results as the sequential program, overheads such as synchronization or communication between threads or processors are needed to synch up or communicate during the computations, in turn leading to a not fully parallelized program.

Ord: 84

Besvarad.

- 2.5** Sometimes we find that a system or application has a *superlinear speedup*. Clearly describe two situations when and why it may occur.

Superlinear speedup, meaning $S(p) \geq 1$, can sometimes occur when parallelizing an algorithm where the sequential algorithm has to keep fetching from global memory while the parallelized version could (assuming enough processors or threads) store parts of the problem in each processor/threads cache memory, in turn working more efficiently rather than fetching from global memory.

Another could be algorithmic improvements, as some instances of search-through-a-maze algorithms may have a bad performance sequentially, but parallelized could be improved by having multiple processors/threads each have a search path, in turn being faster since one of these many could find it before the sequential algorithm.

Ord: 102

Besvarad.

3.1 Reduction sum

Consider the two functions SumReduction1 and SumReduction2 below:

```
__global__ void SumReduction1(float* input) {
    int i = 2*threadIdx.x
    for (int stride = 1; stride <= blockDim.x; stride *= 2) {
        if (threadIdx.x % stride == 0) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}

__global__ void SumReduction2(float* input) {
    int i = threadIdx.x
    for (int stride = blockDim.x; stride >= 1; stride /= 2) {
        if (threadIdx.x < stride) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}
```

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ SumReduction2 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction1. 

☐ SumReduction1 uses a gather approach and SumReduction2 uses a scatter approach.

☒ SumReduction2 has less control divergence than SumReduction1 

☐ SumReduction1 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction2.

☐ SumReduction1 has less control divergence than SumReduction2.

Rätt. 2 av 2 poäng.

3.2 GPU and CUDA general

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☐ The maximum number of threads that a GPU can handle is the same as the number of cores on the GPU.

☒ In general, the part of the silicon area that is used for implementing processing (e.g. ALUs, logic processing, addition, multiplication,...) is larger on a GPU compared to a CPU. 

☐ Four-dimensional data structures (and higher dimensions) cannot be processed on a GPU.

☐ A CUDA program is a SPMD (Single Program Multiple Data) program. 

☒ A CUDA program may in some cases run exclusively on the GPU with no part of the program running on the CPU. 

Delvis rätt. 0 av 2 poäng.

3.3 Thread coarsening

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☐ Thread coarsening decreases the completion time of a thread

☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program 

☒ Thread coarsening can reduce the number redundant calculations in a CUDA program 

☐ Thread coarsening increases the parallelism in a CUDA program

☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU

Rätt. 2 av 2 poäng.

3.4

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ All the threads in a thread block see the same registers
- ☐ The CPU can access variables in GPU shared memory
- ☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.
- ☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 
- ☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 

Rätt. 2 av 2 poäng.

3.5 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☒ Threads in the same Warp cannot execute different instructions at the same time. 
- ☒ A Thread Block can contain many Warps. 
- ☐ A Thread Block cannot be larger than a Warp.
- ☐ Threads from different Thread Blocks can execute in the same Warp.
- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.

Rätt. 2 av 2 poäng.

3.6

Consider the following small program.

```
for (int m = 0; m < M; m++) {  
    input[m].value += 5;  
    for (int n = 0; n < N; n++) {  
        output[n].value += input[m].value / distance(input[m].position, output[n].position);  
    }  
}
```

If we transform this program into a parallel CUDA program, we may run into some serious performance problems in the CUDA program.

What kind of performance problems may the CUDA program suffer from? How could we modify the small program above so that it is still functionally equivalent and so that the corresponding CUDA program does not suffer from any serious performance problems?

One issue noted is that e.g. for the diagonal, multiple threads will perform computation on the same values repetitively, in turn leading to incorrect values.

One way to manage this is to format so that warps work on the arrays in row wise to ensure efficiency and not overlap computations on the same elements.

Ord: 54

Besvarad.

3.7

Explain how sparse matrices are stored when using the ELL format. Also explain the potential performance advantages for a CUDA program when using the ELL format compared to the CSR (compressed sparse row) format. Also, explain how sparse matrices are stored when using the COO (coordinate list) format. Also, explain why we sometimes would like to combine the ELL and COO formats when using CUDA. Give an example of how the ELL and COO formats can be combined when storing a sparse matrix.

The ELL format uses two arrays to store the non-zero values of each row in a matrix and their indices, respectively. Each row is the same length, and is padded to be same length if necessary. Compared to CSR, which similarly stores the non-zero values in an array, ELL is more efficient for Sparse Matrix-Vector computations (SpMV) as it keeps a coalesced structure of the relevant values and their indices.

COO stores sparse matrixes using a tuple: (row, col, value), which in turn is useful for matrix creation or irregular matrices, but is less efficient when it comes to SpMV computation.

The optimal combination, for handling all sorts of sparse matrixes, is to combine ELL and COO - essentially using ELL for the "regular" parts of the matrix and COO for the "irregular parts."

An example is when a sparse matrix has few elements but rows or columns filled with many zeros. This can then be stored using both ELL and COO combined:

Example:

[1, 2]

[3, 4, 5, 6]

[7, 8, 9, 10]

[11, 12]

This can be stored as:

ELL:

Row. [[1,2], [3,4], [7,8], [11, 12]]

Indices: [[0, 1], [0, 1], [0, 1], [0, 1]]

COO:

(row, col, val):

[(1, 2, 5), (1, 3, 6)]

Ord: 207

Besvarad.

3.8

How can the programmer make the GPU put a variable in the cache? Where is such a variable stored when it is not in the cache?

When launching the kernel, the values placed in the parameters will be stored in the cache when the GPU launches. Before that they are stored in the CPU cache or local memory.

Ord: 32

Besvarad.

3.9 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Normally, when working on a histogram, threads have to contend for access of the global histogram bins, in turn leading to a serialized bottleneck. Privatization solves this by storing private local copies of the histogram for each thread (or warp/thread block) in their local/shared memory.. This in turn avoids having to use atomic operations to manage and update the global memory each time a thread finishes. When the computations are finished, the local histograms are merged (reduced) to a final global histogram.

This improves the performance by firstly avoiding the use of expensive atomic operations, as updating the global memory takes hundreds of cycles per instructions (expensive). By as well using the local/shared memory instead of having to access the global memory each time the thread needs to use the histogram. Lastly, the costly merge back to the global histogram is done once per bin or per thread block, and in turn is less costly than doing it each time a thread finishes computation.

Ord: 164

Besvarad.

3.10 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.

Tiling works by breaking down a large dataset into smaller sub-problems, called tiles. Each tile is then loaded in by a thread block into its shared memory. It then performs computations on it, and returns the results to the global memory. This in turn improves performance since each thread uses the shared memory rather than the global memory, in turn being much more efficient.

We need to use `__syncthreads()` to ensure that all data has been loaded in before computations begin, as well as making sure all the data has been loaded back before loading in a new tile.

Ord: 99

Besvarad.